



# HiLCoE

School of Computer Science  
& Technology

## *CarpoolConnect (Part II)*

**Members** Alazar Gebremedhin  
Delina Fikru  
Henok Mesaye  
Hiyab Yemane  
Luliam Tekle  
Remla Miftah  
Samuel Temesgen

**Course** Object Oriented Software Engineering (CS446)

**Batch/Section** DRB2202/B

**Instructor** Axumawit Tassew

**Term** Spring, 2025

**Date** Aug 29, 2025

## Contents

|          |                                      |          |
|----------|--------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                  | <b>4</b> |
| <b>2</b> | <b>Design Goals</b>                  | <b>4</b> |
| <b>3</b> | <b>Software Architecture</b>         | <b>4</b> |
| 3.1      | Why Clean Architecture? . . . . .    | 5        |
| 3.1.1    | High-Level Package Diagram . . . . . | 7        |
| <b>4</b> | <b>Subsystem Decomposition</b>       | <b>8</b> |
| <b>5</b> | <b>Detail Design</b>                 | <b>9</b> |

## List of Figures

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |    |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Clean architecture. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 5  |
| 2  | Clean architecture. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 5  |
| 3  | A high-level package diagram of <i>CarpoolConnect</i> architected to follow <b>Clean Architecture</b> . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 7  |
| 4  | This diagram provides the architectural overview, showing how the different parts of the system interact. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 8  |
| 5  | This diagram illustrates the foundational business objects of the system. These entities reside in the innermost layer of the Clean Architecture and have no dependencies on other layers. . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 9  |
| 6  | This diagram shows how the Ride entity manages its lifecycle using the <b>State Pattern</b> . The Ride class acts as the context, delegating state-specific behavior to concrete RideState objects. This avoids large conditional statements and makes the system easy to extend with new states. . . .                                                                                                                                                                                                                                                                                                                                          | 10 |
| 7  | This diagram demonstrates the use of the <b>Builder Pattern</b> for constructing complex Ride objects. The RideBuilder provides a fluent API to assemble a Ride, ensuring that it is always in a valid state upon creation. . . . .                                                                                                                                                                                                                                                                                                                                                                                                              | 11 |
| 8  | This diagram details the MatchingEngine, which employs the <b>Singleton Pattern</b> to ensure a single instance and the <b>Strategy Pattern</b> to allow for interchangeable matching algorithms. .                                                                                                                                                                                                                                                                                                                                                                                                                                              | 12 |
| 9  | This diagram showcases how the system interacts with external services. The <b>Observer Pattern</b> is used to decouple the Ride object from actions like sending notifications. The <b>Adapter Pattern</b> is used to isolate the application from specific third-party APIs (e.g., Firebase, Google Maps), adhering to the <b>Dependency Inversion Principle</b> . . . . .                                                                                                                                                                                                                                                                     | 13 |
| 10 | The AuthController acts as the entry point, which then invokes the AuthenticateUserUseCase. This use case is responsible for coordinating with the gateway and repository. Implements the <b>Adapter Pattern</b> : The target interface is ISocialAuthGateway. This is the clean, abstract interface that the use case depends on. It defines a simple method verifyTokenAndGetUserProfile(). The adapters are GoogleAuthAdapter and AppleAuthAdapter. Each one implements the ISocialAuthGateway interface. The adaptees are the third-party libraries (GoogleSignInSDK, AppleSignInAPI), which have specific, incompatible interfaces. . . . . | 14 |
| 11 | This diagram models the passenger-centric use cases. It introduces the RideRequest entity, which represents a passenger's intent before a match is made, and a RealTimeTrackingService to handle location updates. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                       | 15 |
| 12 | This diagram focuses on the driver's actions. It models the Use case classes that interact with the Ride entity and its state pattern implementation (from Diagram 6) to manage the trip lifecycle. .                                                                                                                                                                                                                                                                                                                                                                                                                                            | 16 |
| 13 | This diagram models the backend system for administrators. It introduces the <b>Command Pattern</b> as an effective way to handle various admin actions like verifying a driver or investigating a dispute. This pattern encapsulates each request as an object, allowing for parameterization, queuing, and logging of actions. . . . .                                                                                                                                                                                                                                                                                                         | 16 |
| 14 | This diagram models the high-priority SOS alert system. It uses the <b>Observer Pattern</b> where the SOSAlertService acts as the ISubject. When an alert is created, it notifies all registered Observers, such as a real-time dashboard and an emergency notification dispatcher, ensuring a decoupled and rapid response. . . . .                                                                                                                                                                                                                                                                                                             | 17 |

## List of Tables

- 1 Comparison of Software Architectural Patterns. *Note on scoring:* The overall score is tallied based on a point system assigned to each grade across the five design goals, with a maximum possible score of 20. The mapping is as follows: Excellent = 4 points, Good = 3 points, Fair = 2 points, and Poor = 1 point. . . . . 6

## 1 Introduction

This document provides the detailed technical design and architectural specifications for the CarpoolConnect platform. It is the second part of the project documentation and builds directly upon the problem statement, objectives, and requirements established in the CarpoolConnect (Part I) Project Proposal.

The primary purpose of CarpoolConnect is to create an efficient and reliable community-based transportation platform. It aims to reduce traffic congestion and travel times for urban commuters in Ethiopia by matching drivers with passengers traveling along similar routes. This document outlines the architectural decisions and design principles that will guide the development of a system that is scalable, maintainable, secure, and adaptable to future growth and changes in technology.

## 2 Design Goals

The design of the CarpoolConnect system is driven by several key goals derived from the non-functional requirements. These goals will ensure the platform is robust, user-friendly, and successful in the long term. A primary goal is **maintainability and adaptability**. The system's architecture must be easy to understand and modify, which will allow the development team to add new features, such as the advanced machine learning capabilities planned for Phase II, and to fix issues without introducing new problems. The design will achieve this by supporting a clean separation of concerns, making the codebase simple to navigate for both current and future developers. High **testability** is another critical objective. Every part of the system's core business logic must be verifiable through automated tests. The chosen architecture will isolate these business rules from external elements like the user interface or the database. This separation allows for thorough and efficient testing, which directly leads to higher software quality and fewer bugs in the final product.

The design must also ensure **scalability**. The platform is intended to support a growing community, starting with an initial target of up to 10,000 concurrent users. Therefore, the architecture must allow different parts of the system, especially the computationally-intensive Matching Engine, to scale independently to handle peak demand without a loss in performance. Furthermore, the system will be engineered for **reliability**. It must be consistently available to users, targeting a 99.5% uptime. This requires an ability to handle failures of external services gracefully, ensuring that a problem with a third-party mapping API, for example, does not crash the entire application. All critical data, such as ride completions and user credit balances, must be handled with complete integrity.

Excellent **performance** is essential for a positive user experience. The system must feel fast and responsive, meaning that core user actions, like requesting a ride or viewing a profile, should have response times under 400 milliseconds. The mobile application itself must load quickly, and real-time location updates on the map must appear with minimal delay. Finally, **security** is a top priority. Protecting user data will be enforced through strict measures at every level of the system. This includes encrypting all communication between the app and the backend servers, encrypting sensitive user data stored in the database, and implementing role-based access control for the admin panel to prevent unauthorized access.

## 3 Software Architecture

After evaluating several architectural patterns, the **Clean Architecture** has been selected for the *CarpoolConnect* platform. This modern architectural style provides a clear and disciplined approach to organizing the software.

Clean Architecture organizes the code into a series of concentric circles, each representing a different level of the software. The core business rules and logic reside at the very center, with external details like databases, frameworks, and the user interface on the outer circles. The fundamental principle is the *dependency rule*, which states that all dependencies must point inwards. This means the core business logic knows nothing about the database or the web framework, making it truly independent.

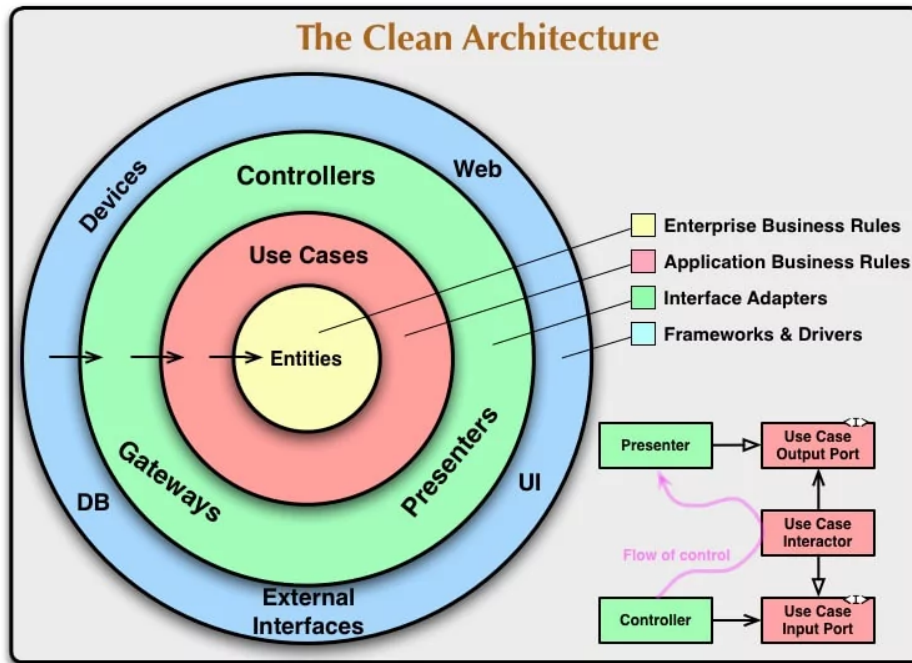


Figure 1: Clean architecture.

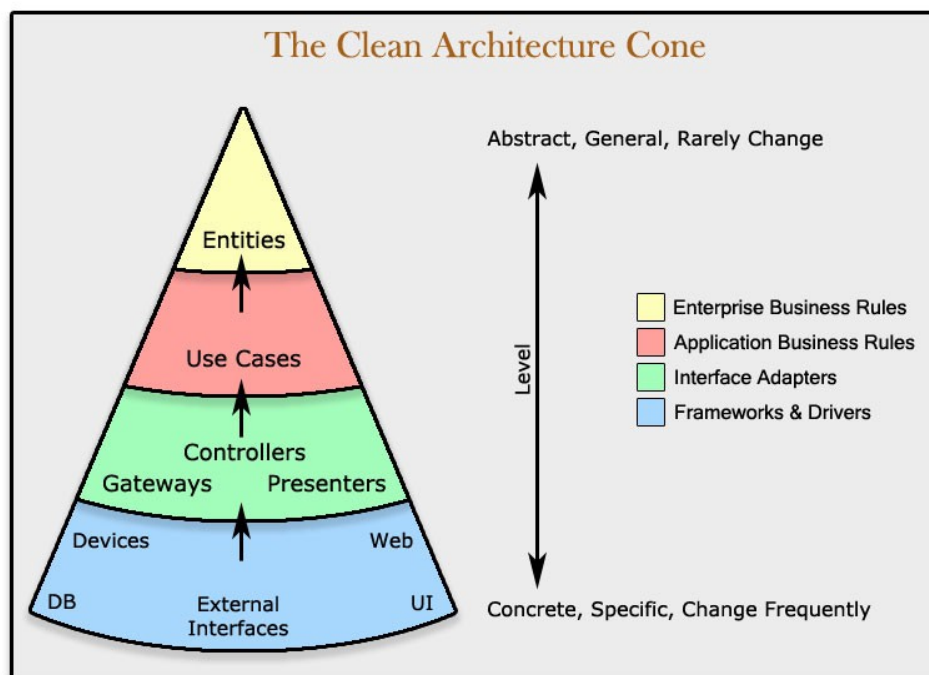


Figure 2: Clean architecture.

### 3.1 Why Clean Architecture?

Clean Architecture directly supports the key design goals of the CarpoolConnect project and is exceptionally well-suited for a startup environment.

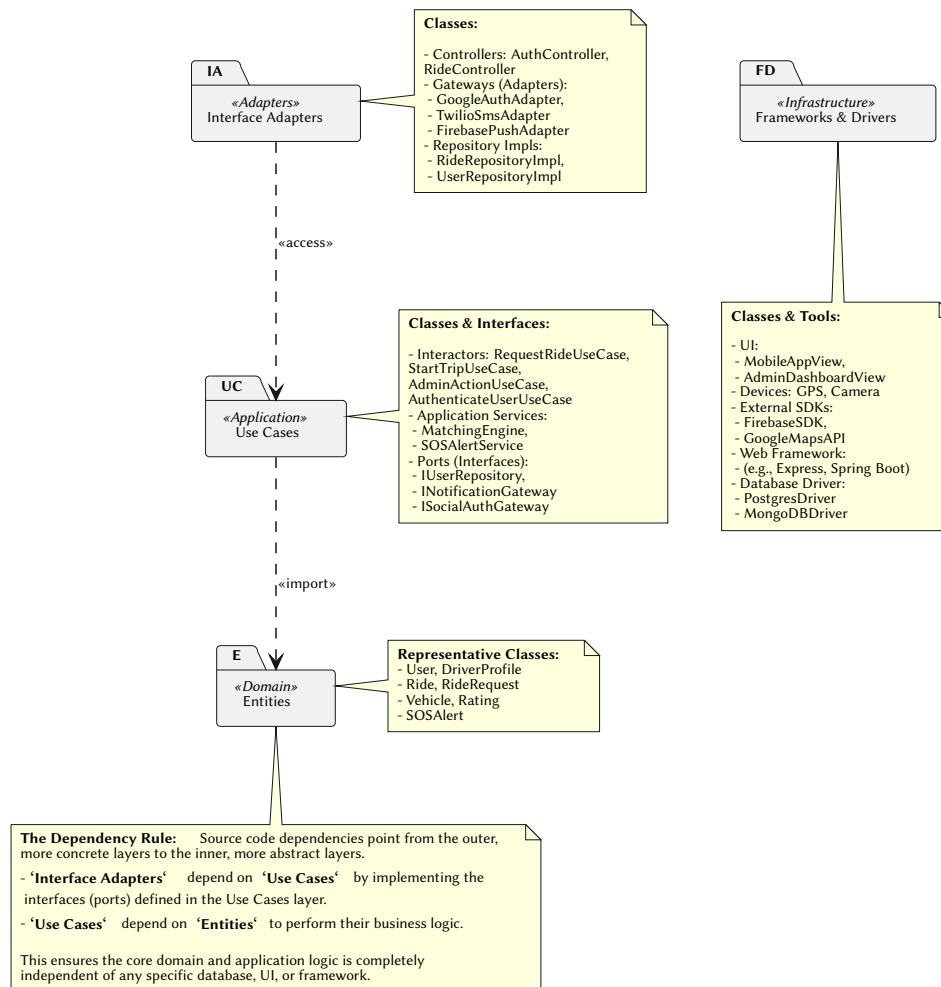
Table 1: Comparison of Software Architectural Patterns. *Note on scoring:* The overall score is tallied based on a point system assigned to each grade across the five design goals, with a maximum possible score of 20. The mapping is as follows: Excellent = 4 points, Good = 3 points, Fair = 2 points, and Poor = 1 point.

| Design Goal                                              | MVC Architecture                               | Layered Architecture                           | Clean Architecture                              |
|----------------------------------------------------------|------------------------------------------------|------------------------------------------------|-------------------------------------------------|
| <b>Maintainability</b><br>(Ease of modification)         | Fair<br><i>Controller can bloat</i>            | Good<br><i>Clear separation of concerns</i>    | Excellent<br><i>Core logic is isolated</i>      |
| <b>Testability</b><br>(Ease of unit testing)             | Fair<br><i>Controller hard to test</i>         | Good<br><i>Business logic is testable</i>      | Excellent<br><i>Core is 100% testable</i>       |
| <b>Low Coupling</b><br>(Component independence)          | Poor<br><i>View/Controller coupled</i>         | Good<br><i>One-way dependency</i>              | Excellent<br><i>Dependency Inversion</i>        |
| <b>Scalability</b><br>(Handling increased load)          | Good<br><i>Scales with web server</i>          | Excellent<br><i>Tiers can scale separately</i> | Excellent<br><i>Components scale separately</i> |
| <b>Simplicity / Learning Curve</b><br>(Ease of adoption) | Excellent<br><i>Widely known, simple start</i> | Good<br><i>Intuitive concept</i>               | Fair<br><i>High initial complexity</i>          |
| <b>Overall Score</b>                                     | <b>12 / 20</b>                                 | <b>16 / 20</b>                                 | <b>18 / 20</b>                                  |

- By separating the core logic of the carpooling application (e.g., how a ride is matched, how credits are calculated) from the implementation details, we can change those details without risk. For instance, we could switch from one database provider to another without rewriting the business rules. **This makes the system highly adaptable and easier to maintain over time.**
- Because the business rules at the center of the architecture have no dependencies on external systems, **they can be easily unit-tested.** We can verify the core functionality of our application without needing to run the entire app, a database, or a web server. This leads to faster development cycles and a more reliable product.
- For a greenfield project like *CarpoolConnect*, it is beneficial to not be locked into specific technologies too early. Clean Architecture allows us to build and test the core application logic before making final decisions on which database, web framework, or UI framework to use. This freedom allows us to choose **the best tools for the job as the project evolves.**
- This architecture forces a focus on the domain: the actual business of carpooling. **By placing these rules at the center, we ensure that the software is built around the core business needs,** not around a particular technology or framework. This is crucial for delivering real value to users and stakeholders.

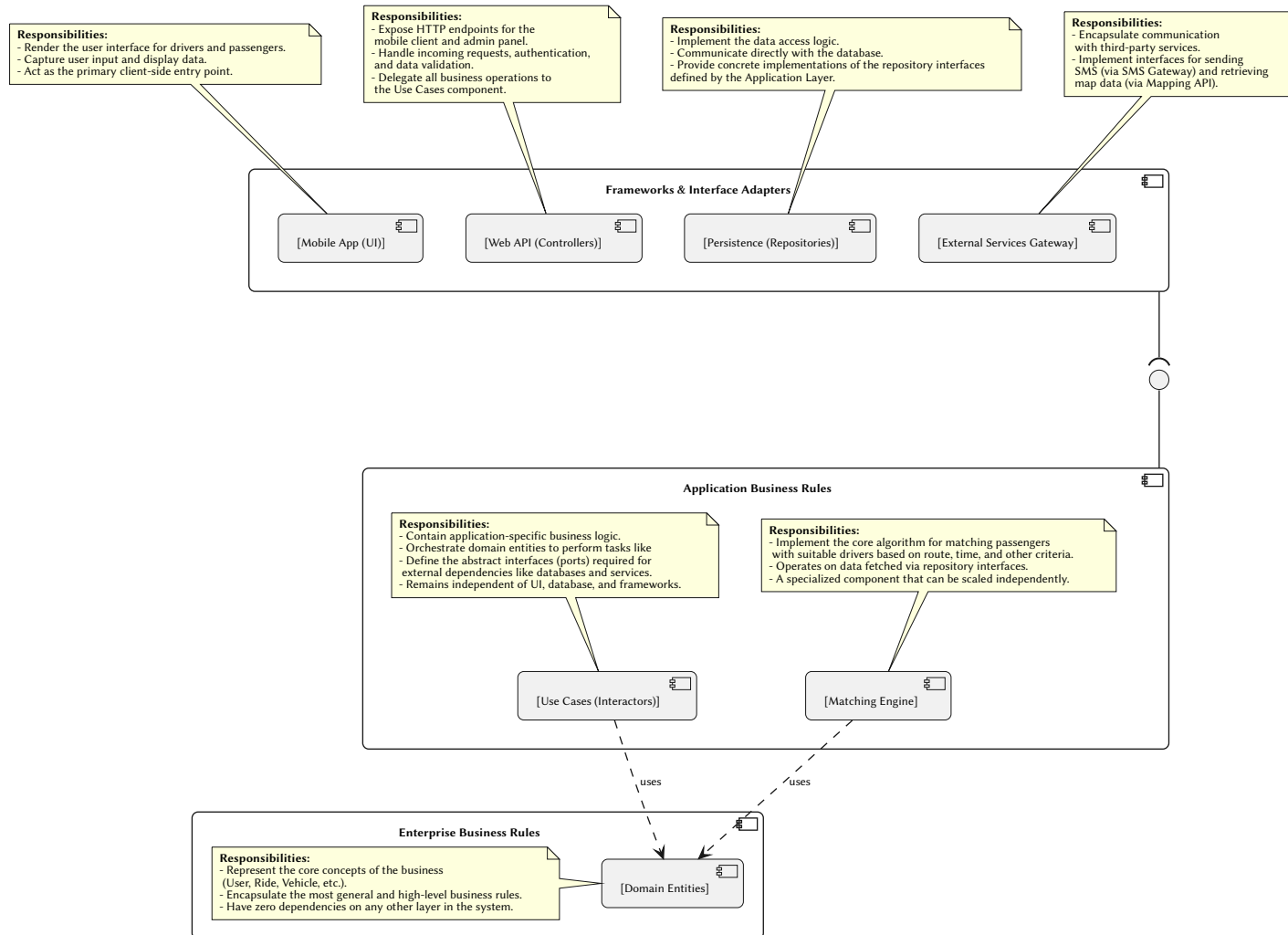
### 3.1.1 High-Level Package Diagram

Figure 3: A high-level package diagram of *CarpoolConnect* architected to follow **Clean Architecture**.



## 4 Subsystem Decomposition

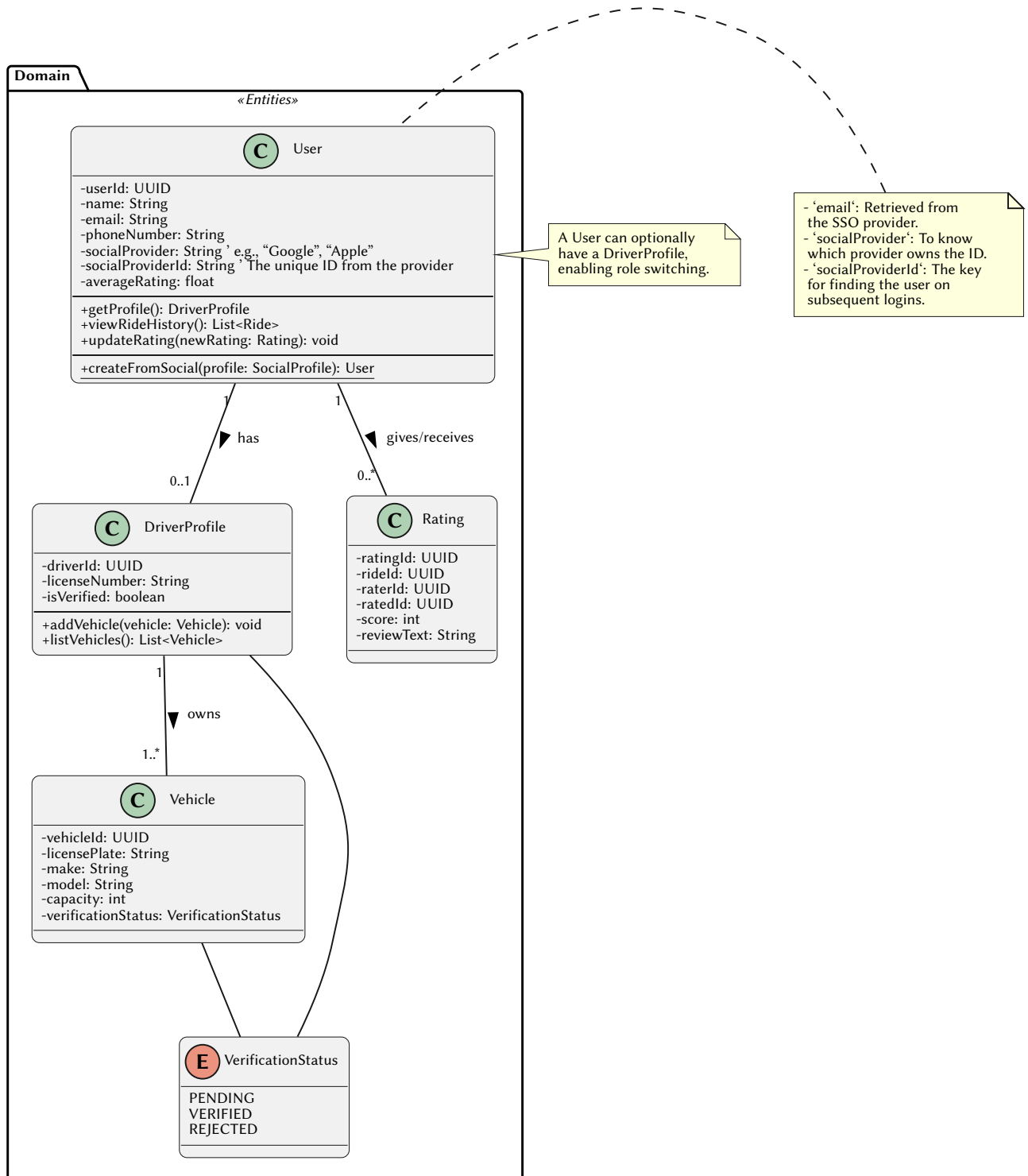
Figure 4: This diagram provides the architectural overview, showing how the different parts of the system interact.





## 5 Detail Design

Figure 5: This diagram illustrates the foundational business objects of the system. These entities reside in the innermost layer of the Clean Architecture and have no dependencies on other layers.



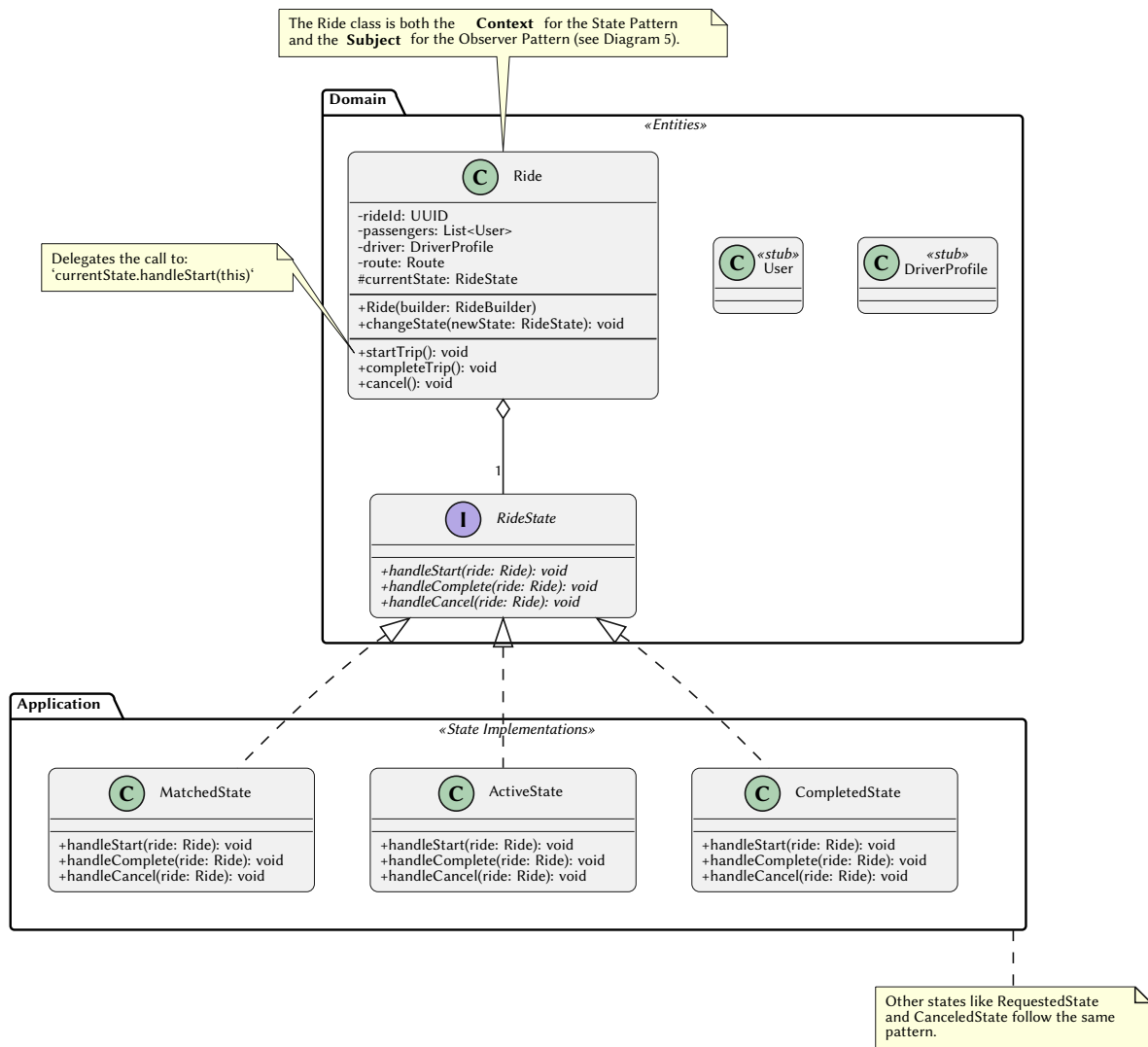


Figure 6: This diagram shows how the Ride entity manages its lifecycle using the **State Pattern**. The Ride class acts as the context, delegating state-specific behavior to concrete RideState objects. This avoids large conditional statements and makes the system easy to extend with new states.

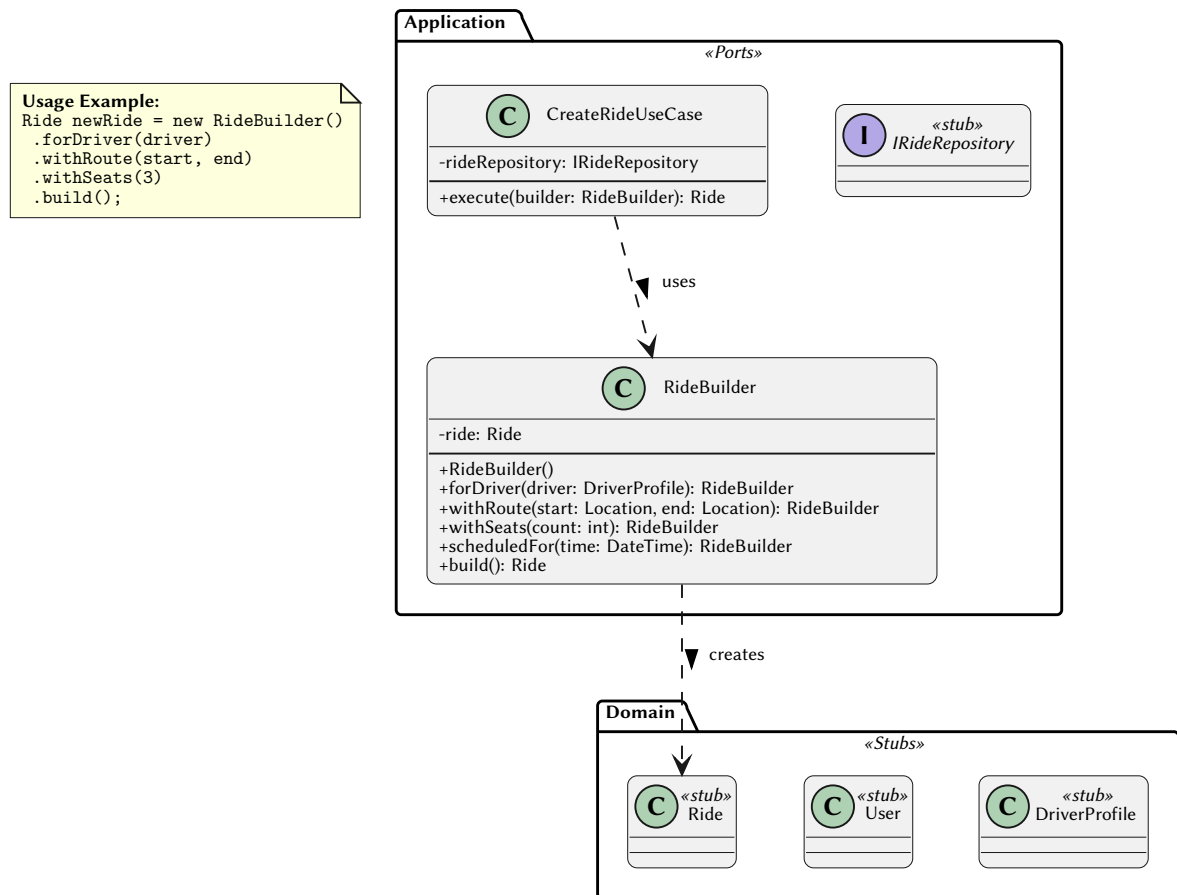


Figure 7: This diagram demonstrates the use of the **Builder Pattern** for constructing complex Ride objects. The RideBuilder provides a fluent API to assemble a Ride, ensuring that it is always in a valid state upon creation.

Figure 8: This diagram details the MatchingEngine, which employs the **Singleton Pattern** to ensure a single instance and the **Strategy Pattern** to allow for interchangeable matching algorithms.

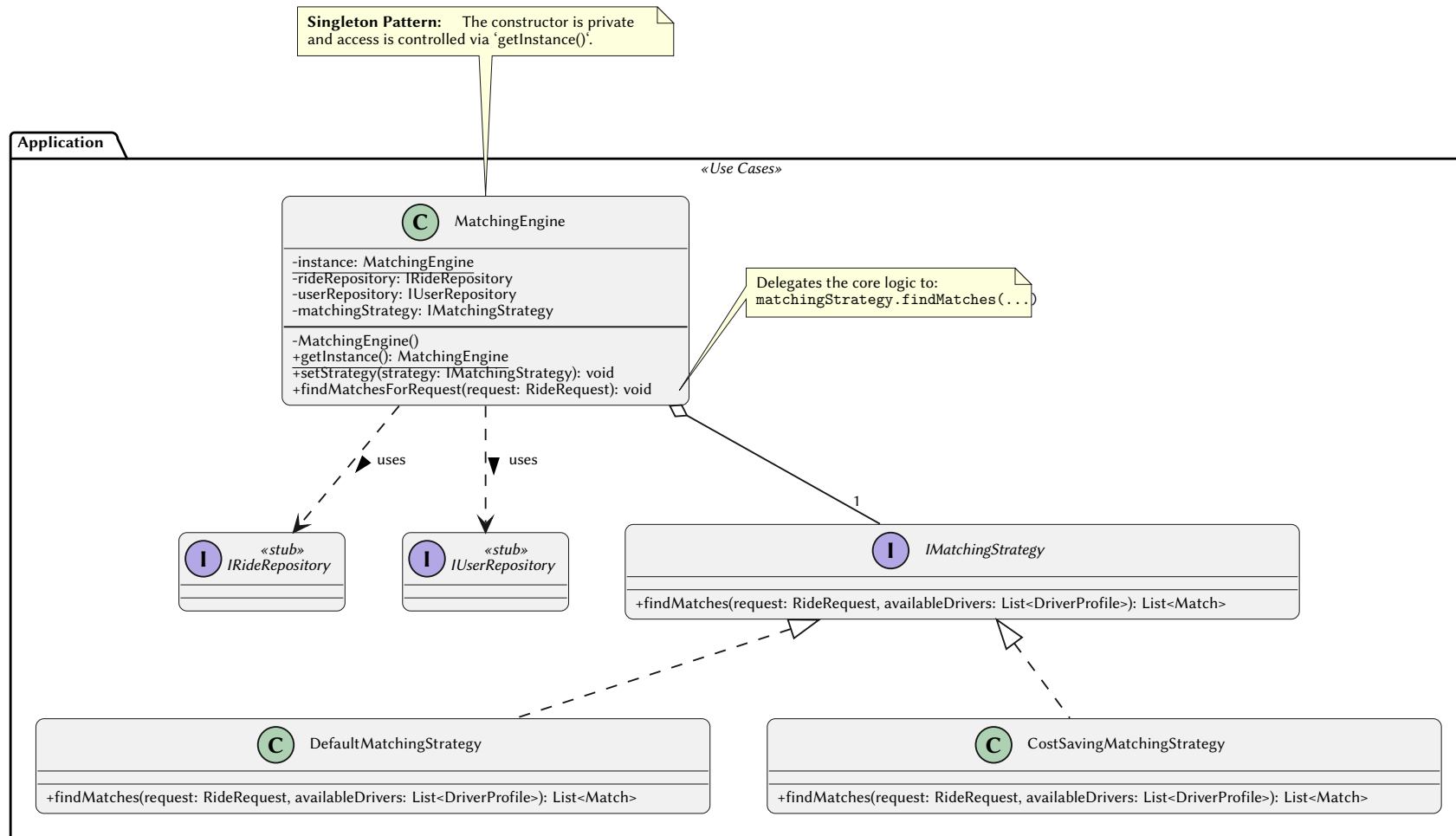


Figure 9: This diagram showcases how the system interacts with external services. The **Observer Pattern** is used to decouple the Ride object from actions like sending notifications. The **Adapter Pattern** is used to isolate the application from specific third-party APIs (e.g., Firebase, Google Maps), adhering to the **Dependency Inversion Principle**.

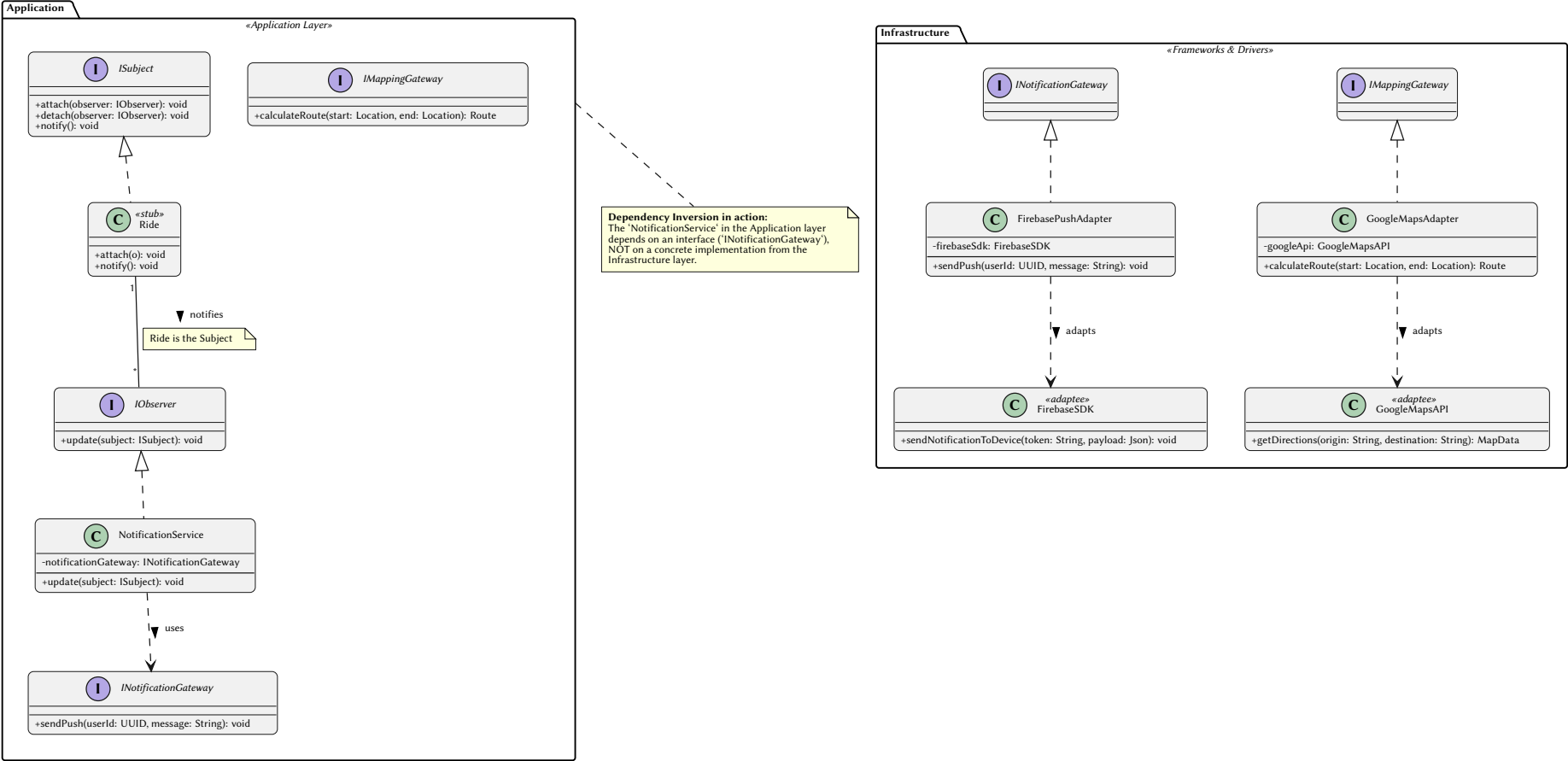
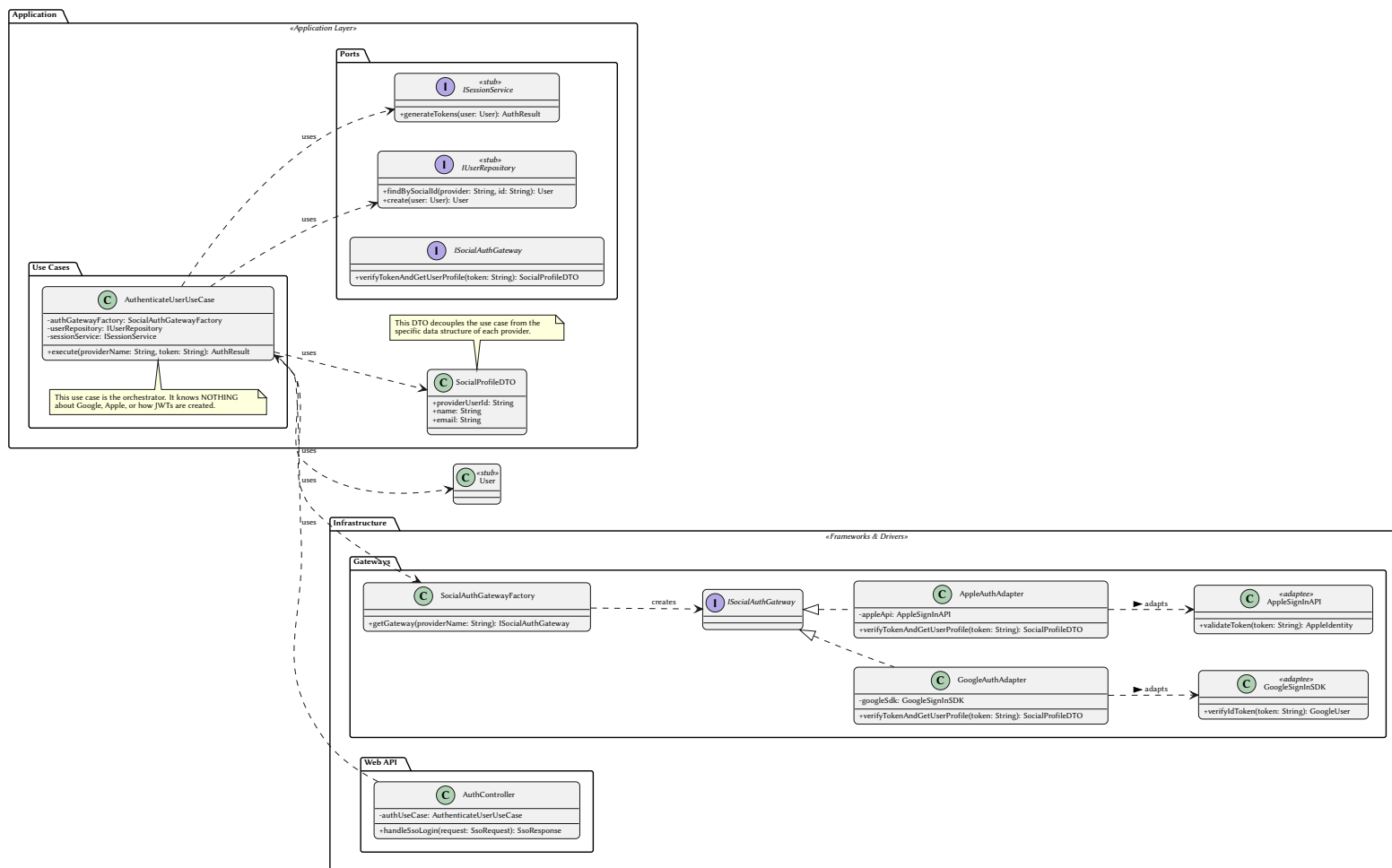


Figure 10: The AuthController acts as the entry point, which then invokes the AuthenticateUserUseCase. This use case is responsible for coordinating with the gateway and repository. Implements the **Adapter Pattern**: The target interface is ISocialAuthGateway. This is the clean, abstract interface that the use case depends on. It defines a simple method verifyTokenAndGetUserProfile(). The adapters are GoogleAuthAdapter and AppleAuthAdapter. Each one implements the ISocialAuthGateway interface. The adaptees are the third-party libraries (GoogleSignInSDK, AppleSignInAPI), which have specific, incompatible interfaces.



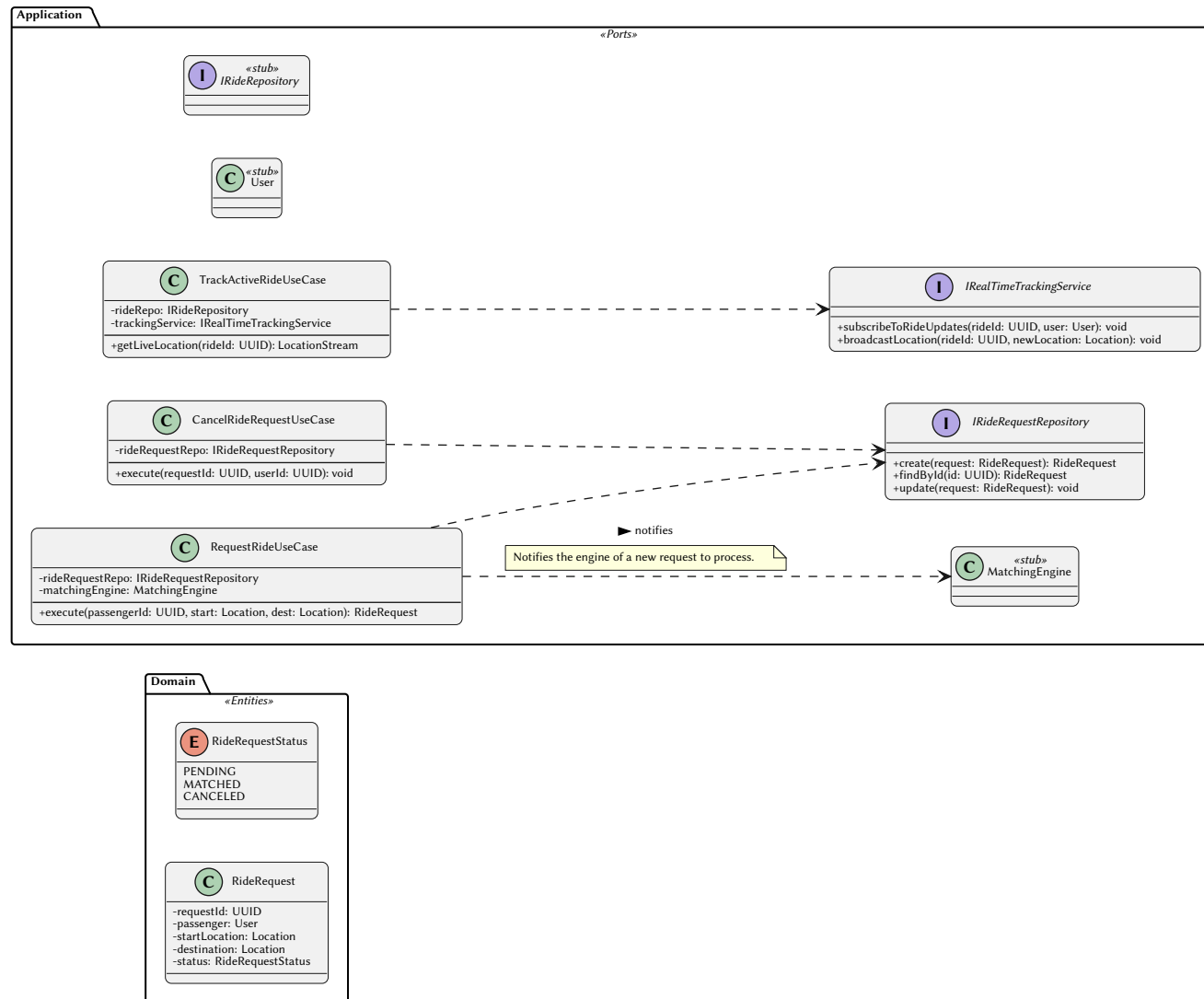


Figure 11: This diagram models the passenger-centric use cases. It introduces the **RideRequest** entity, which represents a passenger's intent before a match is made, and a **RealTimeTrackingService** to handle location updates.

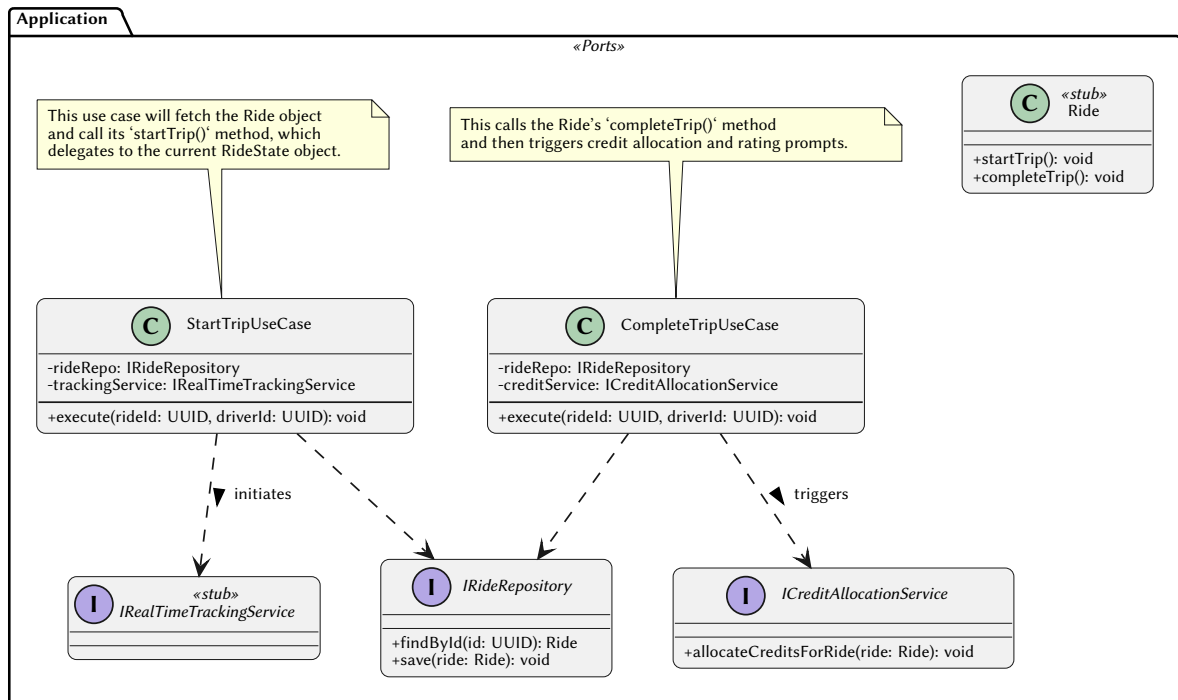


Figure 12: This diagram focuses on the driver's actions. It models the Use case classes that interact with the Ride entity and its state pattern implementation (from Diagram 6) to manage the trip lifecycle.

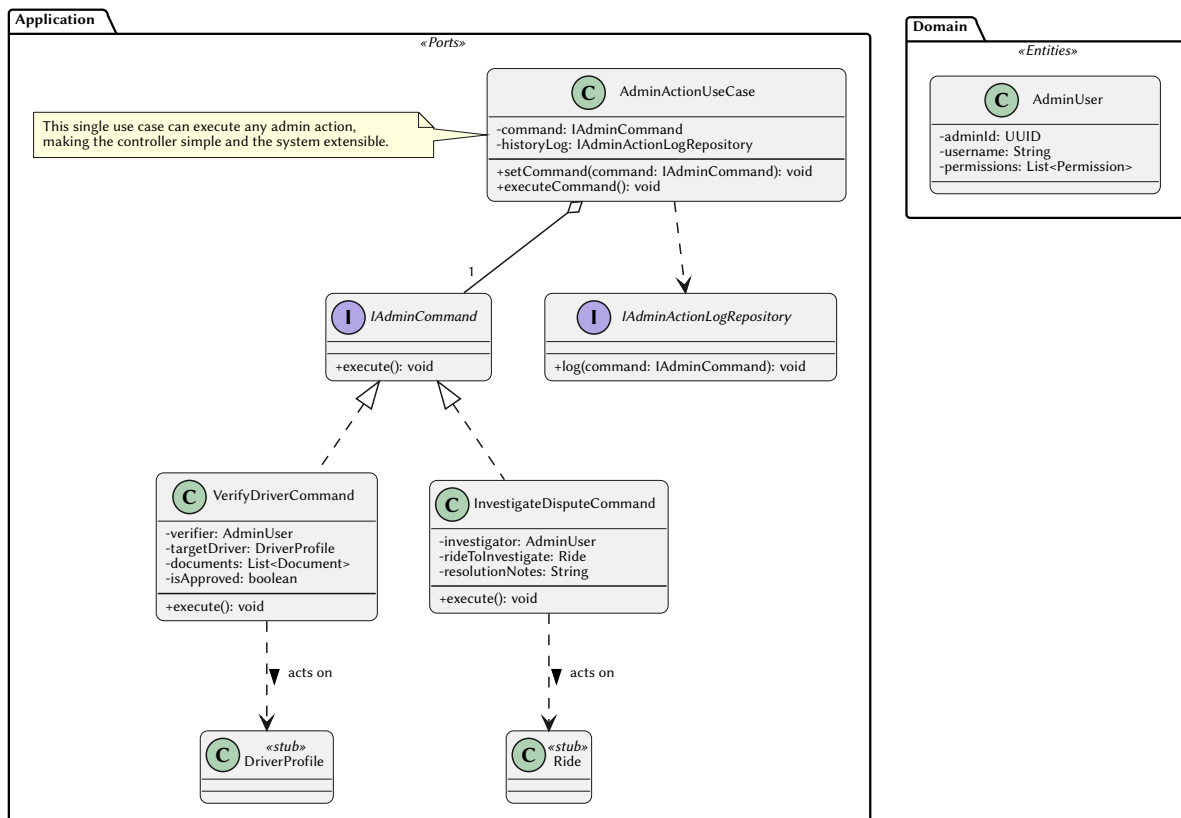


Figure 13: This diagram models the backend system for administrators. It introduces the **Command Pattern** as an effective way to handle various admin actions like verifying a driver or investigating a dispute. This pattern encapsulates each request as an object, allowing for parameterization, queuing, and logging of actions.



